



Laboratorio #1

1.- IntelliJ IDEA

IntelliJ IDEA es un entorno de desarrollo integrado (IDE) desarrollado por JetBrains. Es muy popular entre los desarrolladores por su poderosa funcionalidad y soporte para múltiples lenguajes de programación, aunque es particularmente conocido por su soporte avanzado para Java.

Características Principales:

- **Editor Inteligente:** Ofrece autocompletado inteligente, detección de errores en tiempo real y refactorización de código.
- **Soporte para Frameworks:** Compatible con una amplia gama de frameworks como Spring, Hibernate, Java EE, y también lenguajes como JavaScript, Python, y más.
- **Integración con Herramientas:** Soporte para herramientas de desarrollo como Git, Docker, Kubernetes, y sistemas de compilación como Maven y Gradle.
- **Plugins:** Gran ecosistema de plugins para extender su funcionalidad.

Ventajas:

- **Productividad:** Herramientas avanzadas que aumentan la eficiencia del desarrollador.
- **Personalización:** Altamente configurable según las necesidades del usuario.
- **Comunidad Activa:** Gran comunidad de usuarios y soporte.

En ATI-1 usamos Visual Code. En ATI-2 usaremos IntelliJ IDEA. Es importante que tengamos un comparativo entre diversos IDE para establecer una opinión más amplia de cómo funcionan, adicional a poder comprender las similitudes entre unos y otros, pero particularmente las tendencias del mercado según los componentes técnicos con los cuales trabajamos. URL para instalarlo es: [Descargar IntelliJ IDEA](#)



IMAGEN 1 – IDE IntelliJ

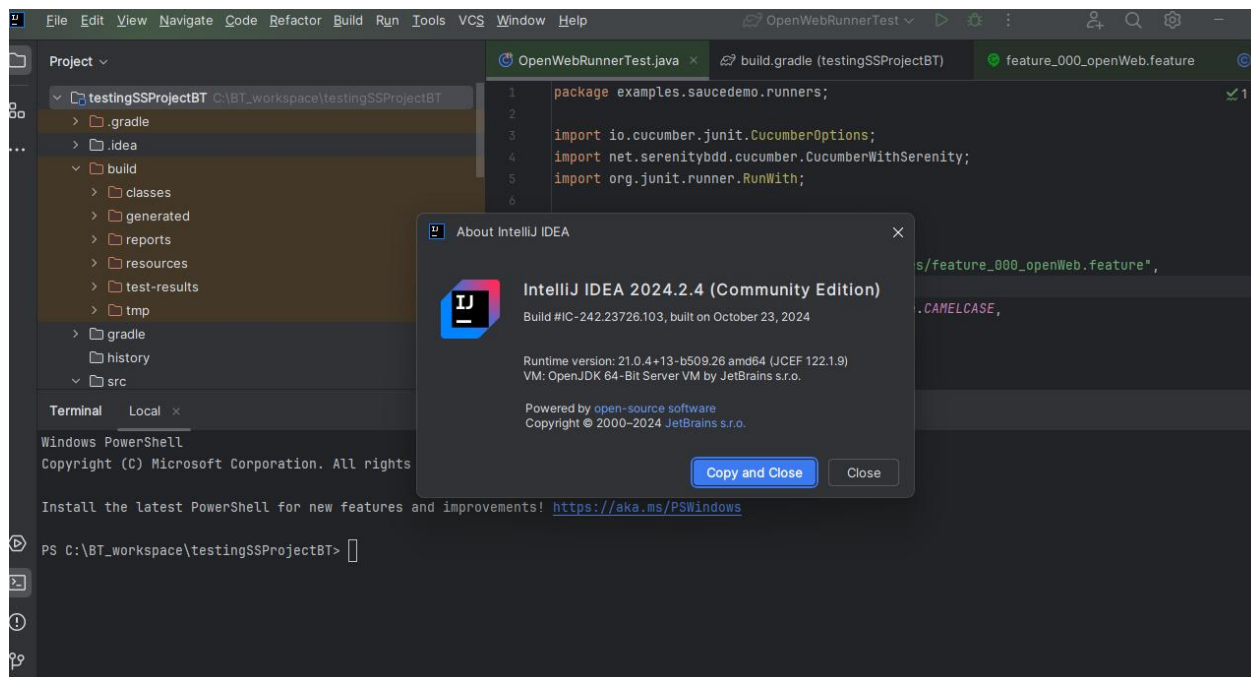


IMAGEN 2 –Comparando IDEs

Característica	IntelliJ IDEA	Visual Studio Code
Desarrollador	JetBrains	Microsoft
Precio	Pago (con versión gratuita limitada)	Gratuito
Lenguajes Soportados	Java, Kotlin, Scala, entre otros	JavaScript, TypeScript, Python, C++, entre otros
Interfaz	Compleja y rica en funciones	Simple y ligera
Extensibilidad	Plugins y extensiones	Extensiones amplias
Depuración	Completa y avanzada	Simple y eficiente
Rendimiento	Potente, pero consume más recursos	Ligero y rápido
Comunidad	Fuerte y dedicada	Gran comunidad y soporte
Integraciones	Integración con otros productos JetBrains	Integración con múltiples herramientas y frameworks

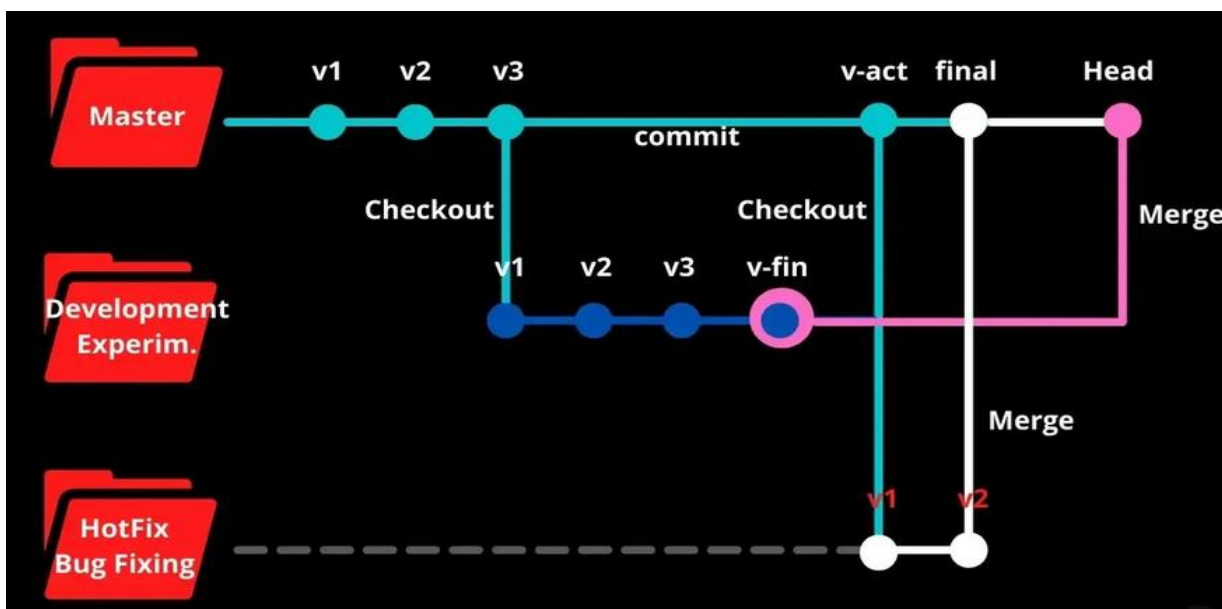
2 .- Git

En ATI -1 trabajamos con Git como herramienta de control de versiones, refresquemos lo aprendido: **Git** es un sistema de control de versiones distribuido que te permite hacer un seguimiento de los cambios en el código fuente a lo largo del tiempo. Fue creado por Linus Torvalds en 2005 para el desarrollo del kernel de Linux, y es utilizado por desarrolladores de software en todo el mundo.

Ventajas:

- **Versiones de Código:** Git guarda un historial completo de los cambios hechos en el código, permitiendo volver a versiones anteriores si es necesario.
- **Colaboración:** Facilita la colaboración entre múltiples desarrolladores, permitiendo trabajar en paralelo y combinar los cambios de cada uno.
- **Ramas y Fusionado:** Permite crear ramas separadas para desarrollar nuevas funcionalidades o corregir errores, y luego fusionar esos cambios en la rama principal. permite rapidez en la gestión de ramas (branch) y la unificación de diferentes versiones (merge).

IMAGEN 3 – Manejo de ramas





- **Distribuido:** Cada desarrollador tiene una copia completa del repositorio, lo que permite trabajar de manera independiente y offline.

Comandos Principales:

- **Commit:** Guardar una instantánea del estado actual del código.
- **Push:** Subir tus cambios a un repositorio remoto (por ejemplo, en GitHub).
- **Pull:** Descargar cambios del repositorio remoto a tu copia local.
- **Branch:** Crear una nueva rama para desarrollar características o solucionar errores sin afectar el código principal.
- **Merge:** Combinar cambios de diferentes ramas en una sola.

Git es esencialmente la columna vertebral del desarrollo colaborativo y la gestión de proyectos de software modernos.

Políticas para el manejo de rama:

Las políticas de manejo de ramas son cruciales para mantener la coherencia, calidad y eficiencia en un proyecto de desarrollo de software, de la misma forma de manejo de versiones basada en las entregas. En ATI2 manejaremos una rama por cada reto que lo requiera. Además de repositorios para el proyecto final, por lo cual es sumamente importante recordar y establecer el convenio del manejo del código de antemano:

1. Ramas Principales:

- **main o master:** La rama principal que contiene la versión estable del código.
- **develop:** Una rama de desarrollo donde se integran nuevas características antes de fusionarlas con main.



2. Ramas de Funcionalidad:

- **Características** (feature/): Ramas dedicadas a desarrollar nuevas características.
- **Corrección de Errores** (bugfix/): Ramas para corregir errores detectados.

3. Ramas de Lanzamiento:

- **release/**: Ramas usadas para preparar versiones finales antes de lanzar.

4. Ramas de Hotfix:

- **hotfix/**: Ramas para aplicar correcciones rápidas a la versión en producción.

5. Ramas de funcionalidad (por programador)

Convención recomendada: [usuario]/[tipo]/[descripcion-breve]

- **usuario**: nombre o alias del programador
- **tipo**: feat (funcionalidad), fix (corrección), poc (prueba de concepto), spike (investigación técnica)
- **descripcion-breve**: resumen de la funcionalidad o cambio

Integración de Git en IntelliJ IDEA

1. **Instalar Git**: Asegúrate de tener Git instalado en tu computador. IntelliJ IDEA puede detectarlo automáticamente.
2. **Configurar Git**: Ve a Configuración (o Preferencias en macOS) y selecciona Control de Versiones > Git. Aquí puedes especificar la ruta del ejecutable de Git si no se detecta automáticamente.
3. **Proyecto existente**: Si ya tienes un proyecto, ve a Archivo > Nuevo > Proyecto desde Control de Versiones y selecciona el repositorio remoto.
4. **Nuevo proyecto**: Si estás creando un nuevo proyecto, puedes habilitar el control de versiones con Git seleccionando VCS > Habilitar Integración de Control de Versiones y eligiendo Git.

Plugins recomendados

1. **GitToolBox**: Amplía la integración de Git con funcionalidades adicionales como anotaciones de autoría y más. Este es el más popular y es el que usaremos en ATI-2.



2. **GitFlow**: Facilita el uso de Git Flow, un flujo de trabajo que ayuda a gestionar ramas de desarrollo y lanzamiento.
3. **Gitignore**: Ayuda a gestionar archivos ignorados por Git, como configuraciones locales y archivos temporales.
4. **Git Stash**: Permite guardar temporalmente cambios sin hacer un commit, útil cuando necesitas cambiar de rama rápidamente.

Estos plugins pueden mejorar tu experiencia de desarrollo y facilitar la gestión de tus proyectos con Git en IntelliJ IDEA.

3.- GitHub

GitHub como sabemos es la plataforma de desarrollo colaborativo que permite hospedar repositorios del sistema de control de versiones Git. Confirma que tienes tu usuario activo, lo necesitaremos para AT12: URL: <https://github.com/>.

GitHub es una plataforma web basada en Git para el control de versiones y la colaboración de proyectos de software.

Características Principales:

- **Repositorio Remoto**: Aloja y comparte repositorios de código.
- **Control de versiones**: Permite gestionar y registrar cambios en el código.
- **Colaboración**: Facilita la colaboración entre desarrolladores mediante pull requests y revisiones de código.
- **Integración Continua**: Herramientas para pruebas automáticas y despliegue continuo.
- **Documentación**: Mantiene documentos como README y Wikis.

Funcionalidades Principales:

- **Issues**: Gestión de incidencias y tareas.



- **Pull Requests:** Facilita la revisión y fusión de cambios.
- **Actions:** Automatización de flujos de trabajo mediante CI/CD.
- **Projects:** Tableros de proyectos estilo Kanban.

4.- Archivo “.gitignore”

El archivo “.gitignore”, es un archivo de texto que le dice a Git qué archivos o carpetas ignorar en un proyecto. El nombre de este archivo debe incluir un punto al comienzo, y al editarlo, en cada nueva línea debe incluir un archivo o carpeta que quieras que sea ignorado.

```
$ echo !debug.log >> .gitignore

$ cat .gitignore
*.log
!debug.log

$ git add debug.log

$ git commit -m "Adding debug.log"
```

IMAGEN 4 –
Ejemplo archivo .gitignore

Aunque .gitignore indica que los archivos .log deben ignorarse, se hace una excepción para debug.log, permitiendo que se agregue y se incluya en el repositorio.

Referencia: <https://www.atlassian.com/es/git/tutorials/saving-changes/gitignore>



5.- Archivo readme.md

Un README es esencial para cualquier proyecto, y debemos crearlo al crear repositorios como paso importante. Para ATI2 será requerido que cada proyecto posea este archivo. Cumpliendo con la siguiente estructura:

1. **Título del Proyecto:** Nombre del proyecto.
2. **Descripción:** Breve descripción del proyecto y su propósito.
3. **Instalación:** Pasos para instalar el proyecto.
Requisitos previos y dependencias.
4. **Uso:**
 - Ejemplos de cómo usar el proyecto.
 - Comandos básicos.
5. **Pruebas:**
 - Instrucciones para ejecutar las pruebas.
 - Comandos y dependencias necesarias para las pruebas.
6. **Despliegue:** Pasos para desplegar el proyecto en producción.
7. **Contribución:**
 - Guía para contribuir al proyecto.
 - Normas de estilo de código y procedimientos de pull requests.
8. **Licencia:** Información sobre la licencia del proyecto.
9. **Contacto:** Información de contacto para preguntas y soporte.

[Editor.md](#) es una herramienta de código abierto que permite la edición de Markdown. Un editor embeddable, lo que significa que puedes integrarlo fácilmente en tu sitio web o aplicación para permitir a los usuarios escribir y visualizar contenido en Markdown. Apoyate en este tool para generar tu README.md.



6.- Mejores practicas

6.1 Nombres típicos para ramas de pruebas de concepto (PoC)

- poc/

Abreviatura de "proof of concept". Muy usada para indicar que la rama contiene código experimental o exploratorio.

- Ejemplo: poc/graphql-integration

- experiment/

Indica que se está probando una idea o tecnología.

- Ejemplo: experiment/new-cache-strategy

- spike/

Término tomado de metodologías ágiles como Scrum, donde un "spike" es una investigación técnica rápida.

- Ejemplo: spike/auth-flow

6.2 Buenas prácticas generales para nombrar ramas

Estructura recomendada:

[tipo]/[id-opcional]-[descripcion-breve]

- tipo: feat, fix, hotfix, poc, spike, etc.
- id-opcional: número de issue o ticket relacionado.
- descripcion-breve: clara y concisa, usando guiones para separar palabras.

6.3 Recomendaciones

- Sé descriptivo pero conciso: usa palabras clave que indiquen claramente el propósito de la prueba.
- Evita espacios o caracteres especiales: utiliza guiones - o barras / para separar palabras.
- Mantén consistencia en el equipo: define una convención y documenta los prefijos aceptados.



Actividad en clase

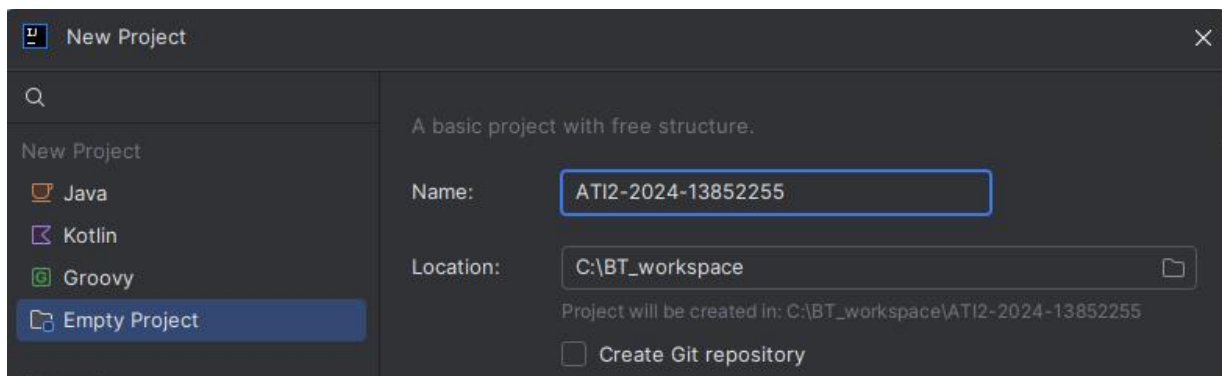
Comparte con tus impresiones en el aula sobre:

- El manejo de git, su importancia en base a la práctica y experiencias anteriores.
- Verifica el archivo “laboratorio_01_RepositoriosGitEjemploProyecto.pdf” del aula virtual y determina la criticidad que implica el manejo de repositorios y las políticas claras en el desarrollo apoyado en este manejo de versiones.
- Las diferencias entre Visual Code e IntelliJ para el manejo de versiones según los pasos prácticos descritos previamente y su experiencia en ATI1.
- Y otros aspectos que consideres interesantes hacer saber a tus compañeros sobre el tema visto.

RETO 02

En ATI-1 creaste un proyecto y se hicieron cambios con comando para subirlo a gitHub desde Visual Code, en este caso crearemos el repositorio desde el IDE IntelliJ usando funciones de la interfaz:

- 2 Instalar IntelliJ: como lo vimos en la práctica solo debes descargar el ejecutable, y seguir las instrucciones del instalador.
- 3 Instalar extensión **GitToolBox** desde marketplace de IntelliJ, **te tocara reiniciar el IDE.**
- 4 **Crear Proyecto:** Abre IntelliJ IDEA.
- 5 Crea el proyecto vacío llamado “ATI2-2024-{tuID}-poc”





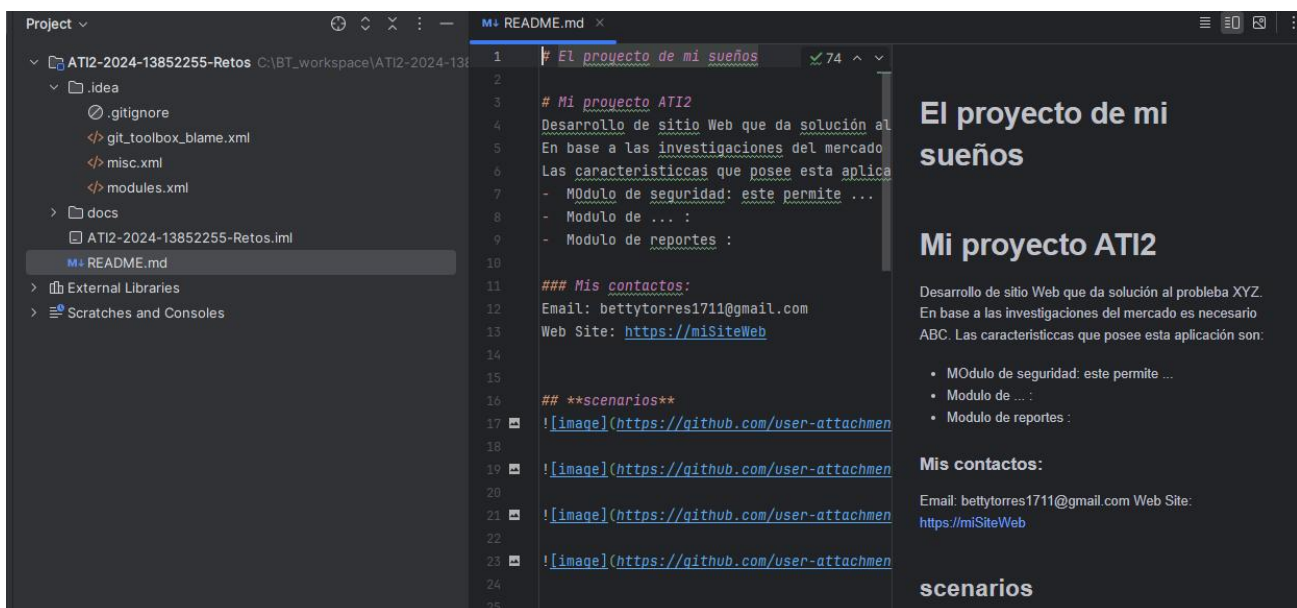
6 Crear una estructura básica para la prueba:

- docs/ -> directorio la documentación
- docs/diagrams -> directorio para mis diagramas
- README.md -> archivo readme del proyecto

7 Actualiza el archivo .gitignore creado por el IDE y agrega **archivos temporales**. Los cuales no requieren seguimiento, apoyate en el ejemplo de la práctica.

8 Editar el archivo README.md agregando título, descripción breve y tu contacto, del proyecto que propondrías desarrollar en este semestre, aprovecha la ventajas de editor.md; sólo funcionalmente.

9 Captura la imagen de la visualización previa en el IDE y los directorios creados.



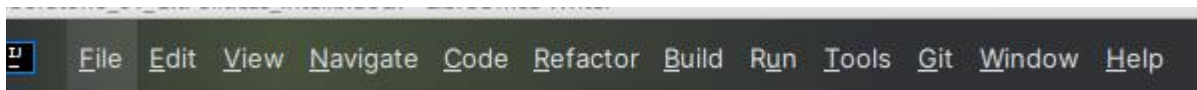
10 **Conectar a GitHub:** Ve a File > Settings (o Preferences en macOS) > Versión Control > GitHub y Añade tu cuenta de GitHub validada. Si tu browser está conectado a gitHub solo tendrás que autorizar el acceso:



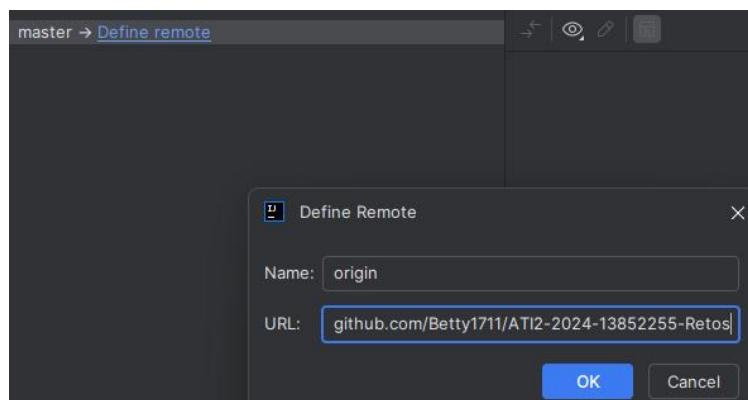
Please continue only if this page is opened from a [JetBrains IDE](#).

[Authorize in GitHub](#)

- 11 Iniciar Git: Ve a VCS > Enable Version Control Integration. Si tienes instalado git el IDE lo reconocerá y tu menú cambiará colocando la opción Git.



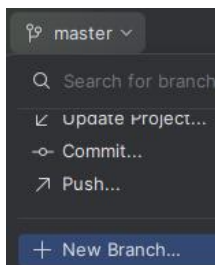
- 12 **Commit Inicial:** Desde la opción Git > Commit
- 13 Guardar los cambios con un commit y un mensaje “Mi repositorios de retos y prácticas de AT12 y sus archivos iniciales”, Colocando el comentario y usando el botón Commit. Lo que hará que localmente tengas tus archivos en el controlador de versiones.
- 14 En gitHub crea el repositorio remoto, Indica el mismo nombre que el de tu repositorio local. Con la descripción “Mi repositorios de retos de AT12”, será privado y agregaras como colaborador a tu revisor. e.g bettytorres1711@gmail.com.
- 15 Observe los archivos creados por defecto.
- 16 Regresando a IntelliJ usa el menú y realiza un **Push a GitHub**: desde Git > Push. Solicitar el URL del repositorio creado.





17 Confirma con el botón “Push”.

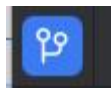
18 Desde IntelliJ usa la rama master y crea la rama “reto2”.



19 En el directorio *doc* coloca el documento de políticas de versiones de ATI1 (incluso en base a lo convenido con tu equipo de trabajo y las experiencias anteriores o el ejemplo visto en clases), en formato pdf, debemos refrescar y tener al día esta información. Actualizarlo de ser necesario en base a tus experiencias, coloca tus propuestas de la nomenclatura de las ramas de un proyecto.

20 En git a GitHub y verifica que tu proyecto fue creado con las características establecidas en la rama master y sus archivos.

21 Identifica las actualizaciones usando el plugin



22 Formaliza tu entrega del **reto 2** en el aula virtual.

Entregable en el aula virtual:

- 1- Print screen sincronización IntelliJ con git.
- 2- URL del repositorio con la rama reto2.

En el repositorio:

- 3- PDF Políticas de manejo de rama y versiones actualizadas.
- 4.- Adicionalmente, el readme, según las pautas vistas, con tu propuesta de proyecto ATI2 (considere las opciones del documento “ATI2-2025-2-LineamientosProyecto”).

Éxitos!

"La tecnología está en constante evolución, y con ella, las oportunidades y desafíos. Adaptarse a estos cambios no solo es crucial para mantenerse relevante, sino también para innovar y liderar en tu campo. La capacidad de adaptarse es la diferencia entre aquellos que prosperan y aquellos que quedan atrás." - Satya Nadella, CEO de Microsoft